

3. Key Steps on How Provider is Used in Flutter

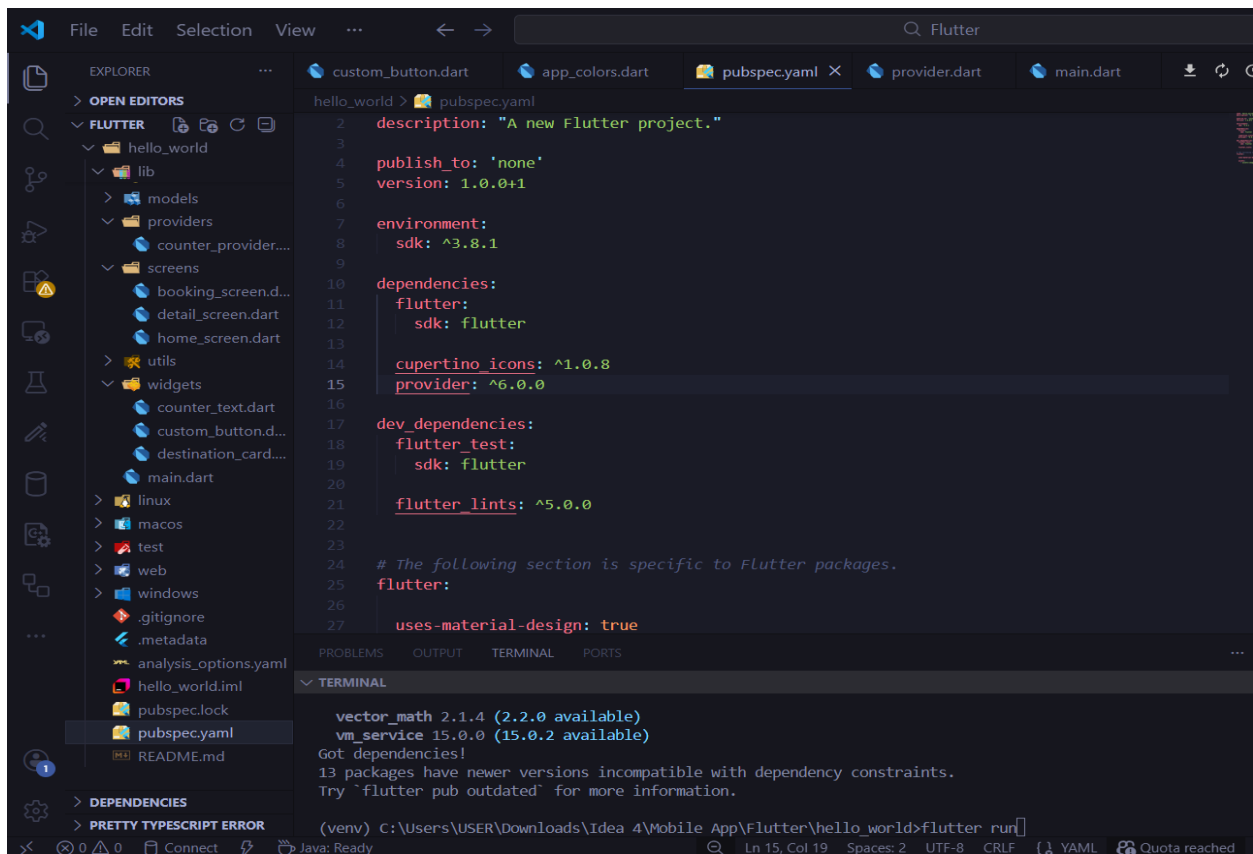
Provider is one of the most commonly used state management solutions in Flutter. It allows data (state) to be shared efficiently across the widget tree and rebuilds only the necessary widgets when the state changes.

Below are the detailed steps explaining how Provider is used in a Flutter application.

1. Adding Dependency

The first step is to add the Provider package to the Flutter project.

Open the pubspec.yaml file and add the dependency under the dependencies section.



The screenshot shows an IDE with the following components:

- EXPLORER:** A file tree on the left showing the project structure. The 'pubspec.yaml' file is selected under the 'hello_world' directory.
- EDITOR:** The main area displays the 'pubspec.yaml' file with the following content:

```
1 description: "A new Flutter project."
2
3
4 publish_to: 'none'
5 version: 1.0.0+1
6
7 environment:
8   sdk: ^3.8.1
9
10 dependencies:
11   flutter:
12     sdk: flutter
13
14   cupertino_icons: ^1.0.8
15   provider: ^6.0.0
16
17 dev_dependencies:
18   flutter_test:
19     sdk: flutter
20
21   flutter_lints: ^5.0.0
22
23
24 # The following section is specific to Flutter packages.
25 flutter:
26
27   uses-material-design: true
```
- TERMINAL:** The bottom panel shows the output of a Flutter command, indicating that dependencies were successfully resolved and listing some outdated packages.

```
vector_math 2.1.4 (2.2.0 available)
vm_service 15.0.0 (15.0.2 available)
Got dependencies!
13 packages have newer versions incompatible with dependency constraints.
Try 'flutter pub outdated' for more information.
```

After adding the dependency, run the following command in the terminal:

```
flutter pub get
```

This installs the Provider package into the project.

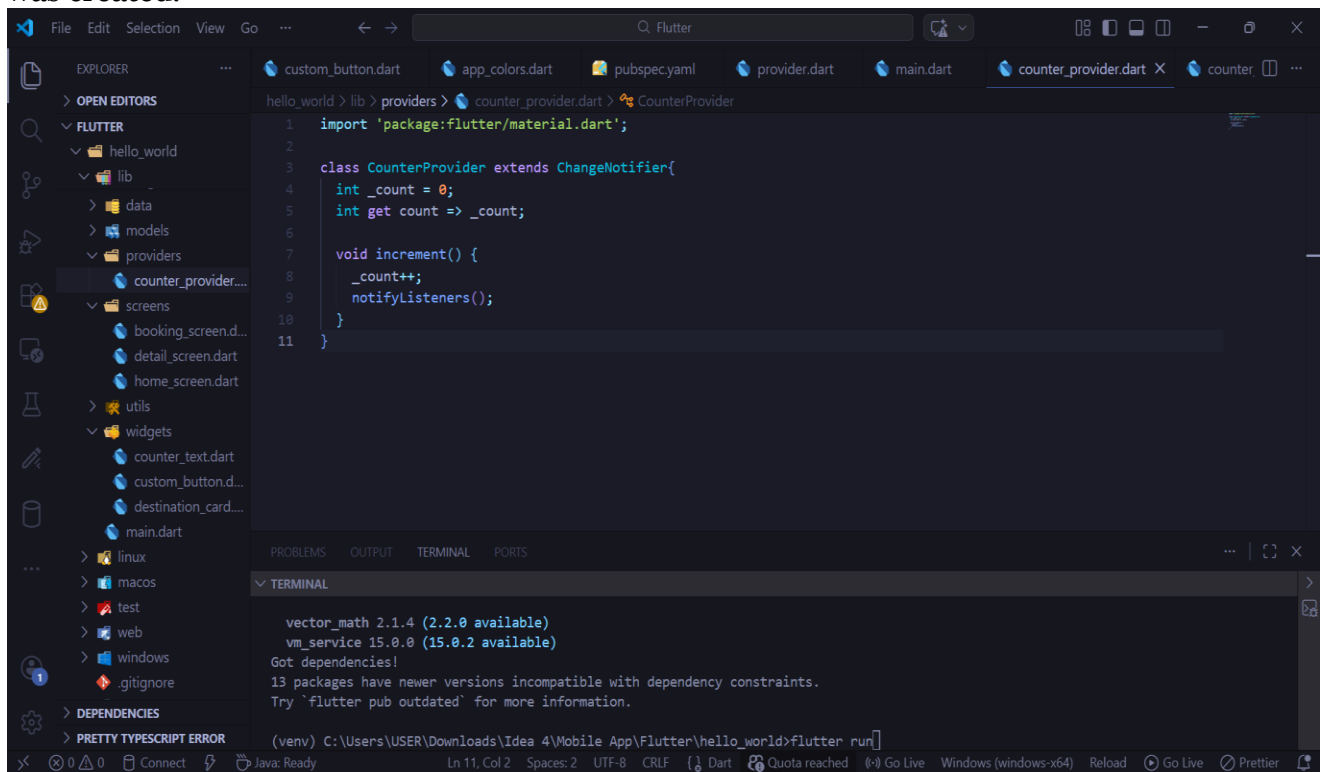
2. Creating a State Class

The next step is to create a state class that holds the data and logic. In Provider, the state class usually extends `ChangeNotifier`.

In this project, a file named:

```
lib/providers/counter_provider.dart
```

was created.



Explanation:

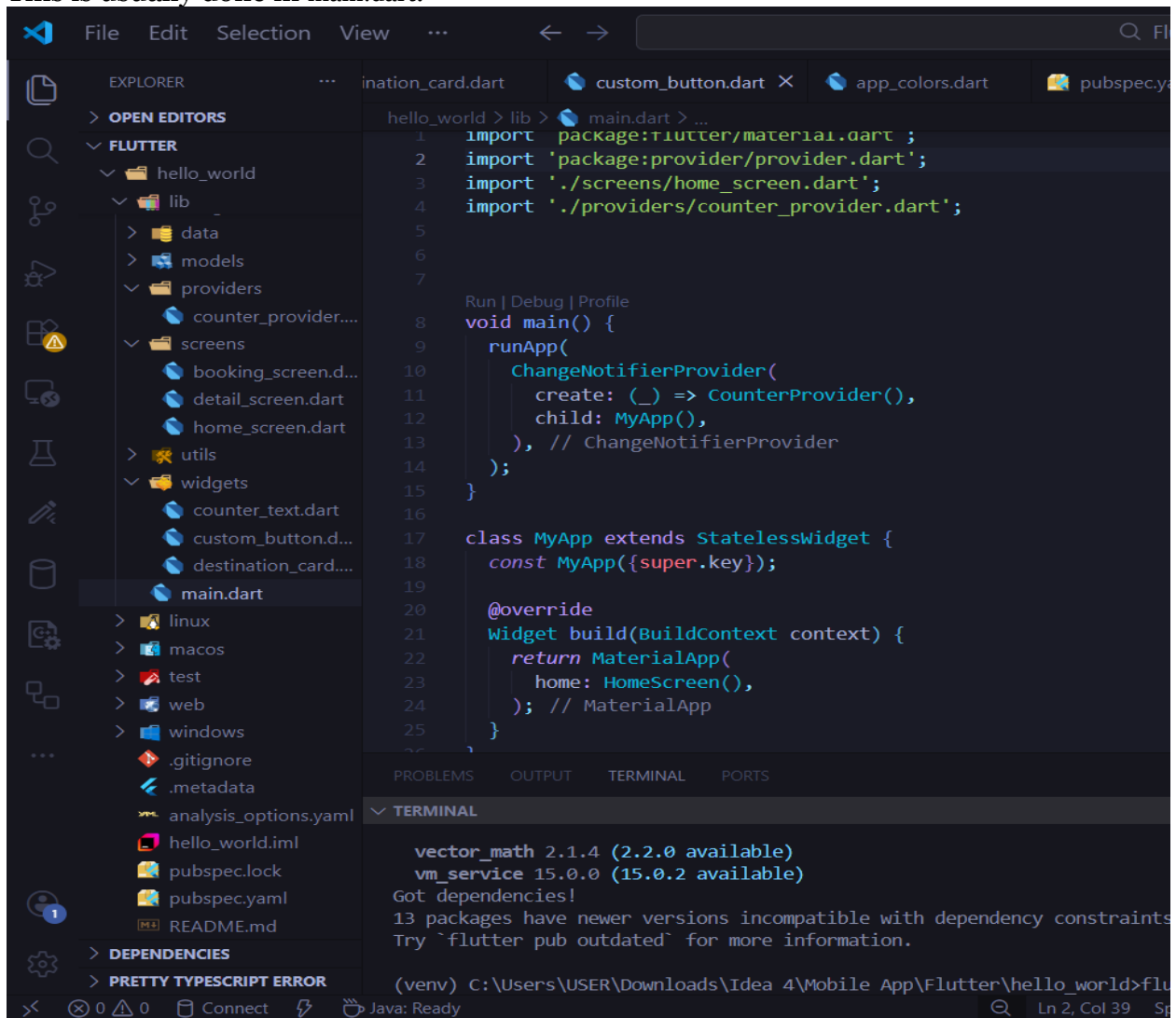
- `_count` is a private variable that stores the counter value.
- `count` is a getter used to access the value.
- `increment()` increases the counter value.
- `notifyListeners()` informs all listening widgets that the state has changed.

This is the core of Provider state management.

3. Providing the State

After creating the state class, it must be provided to the widget tree using `ChangeNotifierProvider`.

This is usually done in `main.dart`.



Explanation:

- `ChangeNotifierProvider` makes the `CounterProvider` available to all widgets below it in the widget tree.
- The `create` function creates an instance of `CounterProvider`.
- `child: MyApp()` allows the entire app to access the provider.

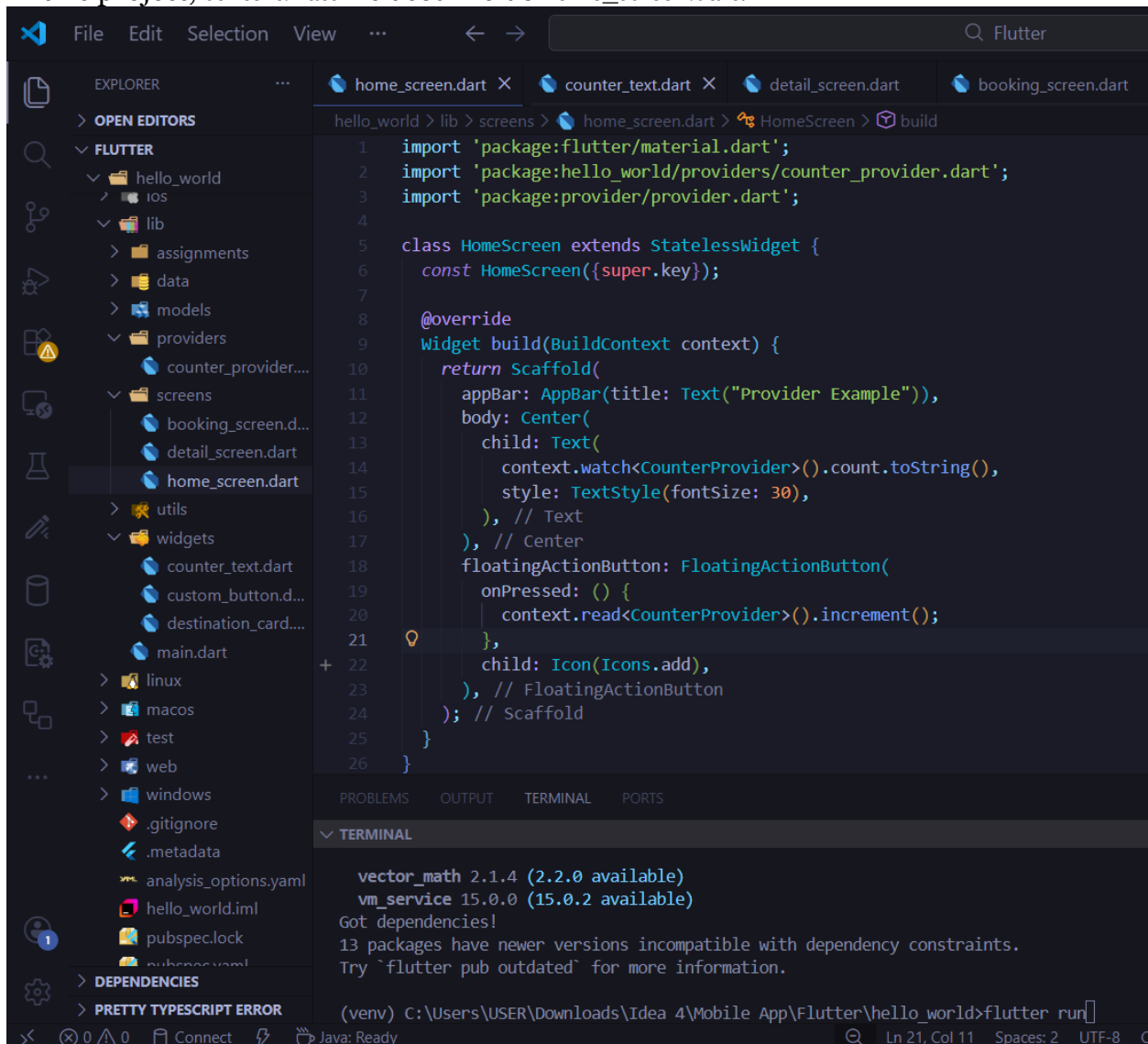
Now the state is globally accessible.

4. Accessing the State

To read the state inside a widget, `Provider` offers multiple methods such as:

- `context.watch<T>()`
- `context.read<T>()`
- `Consumer<T>()`

In this project, `context.watch` is used inside `home_screen.dart`.



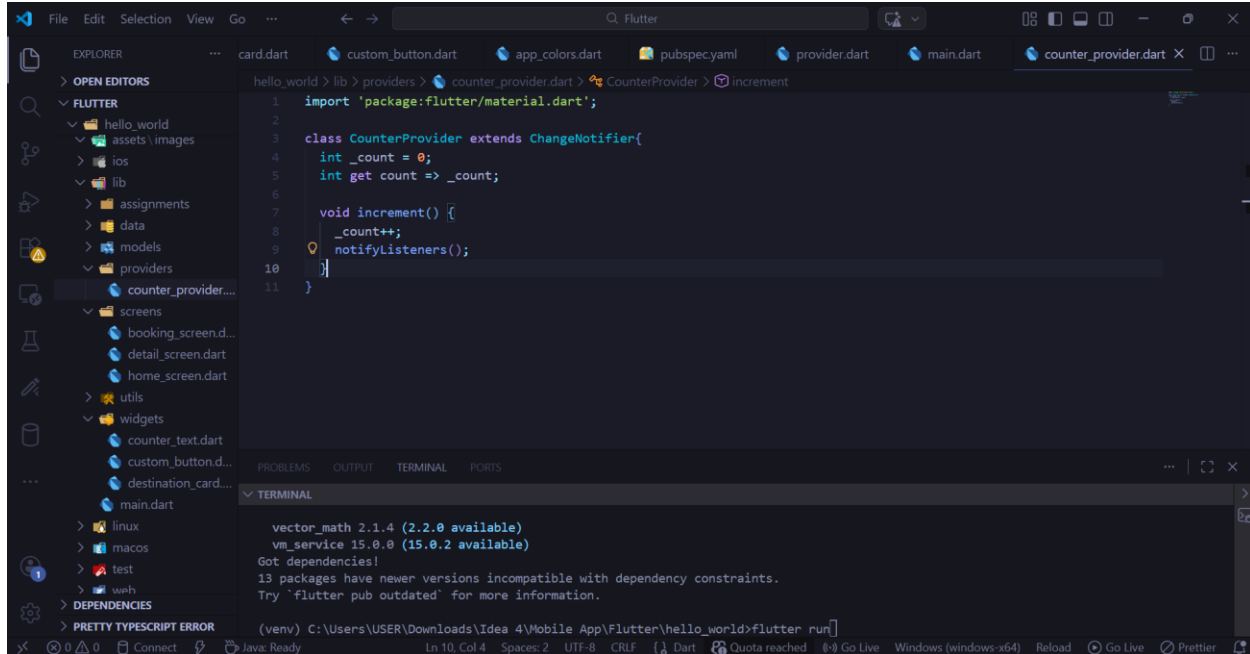
Explanation:

- `context.watch<CounterProvider>()` listens to changes.
- When the state updates, this widget rebuilds automatically.
- The `count` getter is used to display the current value.

5. Updating the State

To update the state, we call a function from the provider class.

In this project, the counter is updated using:



Explanation:

- `context.read<CounterProvider>()` accesses the provider without listening.
- `increment()` updates the `_count` variable.
- After updating, `notifyListeners()` is triggered.

6. How UI Rebuild Happens

The UI rebuild process in Provider works as follows:

1. The user presses the button.
2. The `increment()` function is called.
3. The `_count` variable increases.
4. `notifyListeners()` executes.
5. All widgets using `context.watch()` or `Consumer()` are notified.
6. Those widgets rebuild automatically with the updated value.

This ensures:

- Efficient rebuilding (only listening widgets rebuild)
- Better performance
- Clean separation between UI and business logic